# Operatori bit a bit, altri operatori e sequence point

Marco Cerbella

Lezione 12 – 19 ottobre 2026

## 1 Operatori bit a bit

Gli operatori bit a bit permettono di manipolare i singoli bit di un byte o di un'unità di dati più grande.

| Operatore | Significato | Esempio | Risultato (per ogni posizione di bit) |
|---|---|---|---|
| `&` | AND bit a bit | `x & y` | 1 se 1 sia in $x$ sia in $y$; 0 se 0 in $x$ o in $y$, o in entrambi |
| `\|` | OR bit a bit | `x \| y` | 1 se 1 in $x$ o in $y$, o in entrambi; 0 se 0 in entrambi |
| `^` | XOR bit a bit (OR esclusivo) | `x ^ y` | 1 se 1 in $x$ o in $y$ ma non in entrambi; 0 se i due bit sono uguali |
| `~` | NOT bit a bit (complemento a uno) | `~x` | 1 se 0 in $x$; 0 se 1 in $x$ |

**Esempio.** Con `int a = 6;` e `int b = 11;` (mostrati gli ultimi 5 bit, gli altri valgono 0 tranne che in $\sim a$, dove valgono 1):

| Espressione | Schema di bit |
|---|---|
| `a = 6` | `00110` |
| `b = 11` | `01011` |
| `a & b` | `00010` |
| `a \| b` | `01111` |
| `a ^ b` | `01101` |
| `~a` | `11001` |

Un uso tipico degli assegnamenti composti bit a bit è la *maschera*:

```
1 a &= 0xFF;     // equivale a a = a & 0xFF; azzera tutti i bit tranne gli otto
      meno significativi
```

### 1.1 Rappresentazione in memoria e nel processore

In memoria i valori sono rappresentati in *little endian*; nel processore gli stessi valori vengono tradotti in *big endian*. Tutte le operazioni nel processore sono quindi eseguite in big endian e, quando il risultato viene riscritto in memoria, viene salvato in little endian.

## 2 Operatori di shift

Gli operatori di shift traslano lo schema di bit dell'operando sinistro del numero di posizioni indicato dall'operando destro.

Gli operandi degli operatori di shift devono essere interi. Prima dello shift vengono eseguite su entrambi le promozioni intere. Il valore dell'operando destro non deve essere negativo e deve essere minore della larghezza (in bit) dell'operando sinistro dopo la promozione intera.

**Esempio.**

```
1 unsigned long n = 0xB,   // schema di bit: 0 ... 0 0 0 1 0 1 1
2               result = 0;
3 result = n << 2;         // 0 ... 0 1 0 1 1 0 0
4 result = n >> 2;         // 0 ... 0 0 0 0 0 1 0
```

**Osservazione.** Uno shift a sinistra di $y$ posizioni equivale a moltiplicare l'operando sinistro per $2^y$; uno shift a destra di $y$ posizioni equivale a dividere per $2^y$ (divisione intera): $x/2^y$.

## 3 Altri operatori

In C ci sono altri sei operatori che non rientrano nelle categorie viste.

| Operatore | Significato | Esempio | Risultato |
|---|---|---|---|
| `()` | chiamata di funzione | `log(x)` | passa il controllo alla funzione specificata, con gli argomenti specificati |
| `sizeof` | dimensione in byte di un oggetto o di un tipo | `sizeof x` | numero di byte occupati da $x$ in memoria |
| `(tipo)` | conversione esplicita di tipo (cast) | `(short) x` | il valore di $x$ convertito al tipo specificato |
| `?:` | valutazione condizionale | `x ? y : z` | il valore di $y$ se $x$ è vero (diverso da 0), altrimenti il valore di $z$ |
| `,` | valutazione sequenziale | `x, y` | valuta prima $x$, poi $y$; il risultato è il valore di $y$ |

(gli altri due operatori, di selezione dei membri e di indicizzazione, sono già nella tabella di precedenza.)

### 3.1 Operatore condizionale

L'operatore condizionale è detto anche operatore *ternario*, perché è l'unico che ha tre operandi:

`condizione ? espressione1 : espressione2`

L'operazione valuta prima la condizione. Poi, a seconda del risultato, valuta una delle due espressioni alternative. Dopo la valutazione della condizione c'è un *sequence point*. Se il risultato è diverso da 0 (la condizione è vera) viene valutato *solo* il secondo operando, `espressione1`, e l'intera operazione assume il suo valore. Se invece la condizione è 0 (falsa) viene valutato solo il terzo operando, `espressione2`, e l'operazione assume il suo valore.

**Esempio.**

```
int a = 6;
int b = 12;
int c;
c = b > a ? 100 : 150;      // c diventa 100
c = a == b ? 100 : 150;     // c diventa 150
```

Il primo operando, la condizione, deve avere tipo aritmetico. Se entrambe le espressioni alternative hanno tipo aritmetico, il tipo del risultato è quello che si ottiene applicando agli operandi le usuali conversioni aritmetiche:

```
int a = 6;
int b = 12;
int c;
c = (b > a) ? 100 : 150LL;   // l'intera espressione ha tipo long long
```

### 3.2 Operatore virgola

L'operatore virgola è un operatore binario:

```
espressione1 , espressione2
```

Garantisce un'elaborazione sequenziale: prima viene valutato l'operando sinistro, poi quello destro. Il risultato dell'intera espressione ha il tipo e il valore dell'operando destro. L'operando sinistro viene valutato solo per i suoi effetti collaterali; il suo valore viene scartato.

**Esempio.**

```
int a = 6;
int b = 12;
int c;
c = (a++, a + 7);   // a diventa 7, c diventa 14
```

Il valore dell'intera espressione tra parentesi è 14.

## 4 Sequence point ed effetti collaterali

Una funzione o un'espressione ha un *effetto collaterale* se, oltre a restituire un valore, modifica uno stato o ha un'interazione osservabile con le funzioni chiamanti o con il mondo esterno.

**Definizione.** Un *sequence point* è un punto dell'esecuzione di un programma in cui è garantito che

- tutti gli effetti collaterali delle valutazioni precedenti sono già stati eseguiti, e
- nessun effetto collaterale delle valutazioni successive è ancora stato eseguito.

In altre parole, è un punto in cui tutti gli effetti collaterali sono stati valutati prima di passare al passo successivo.

## 4.1 Attenzione agli assegnamenti

La modifica dell'operando sinistro è un effetto collaterale di un'espressione di assegnamento. Il valore dell'intera espressione di assegnamento è uguale al valore assegnato all'operando sinistro, e l'espressione ha il tipo dell'operando sinistro. A differenza dell'operando sinistro, però, l'espressione di assegnamento *non è un lvalue*.

**Esempio.**

```
int a = 6;
int b = 12;
int c;
c = (b = a);
```

L'espressione `(b = a)` ha valore 6 e, come effetto collaterale, cambia il valore di `b` in memoria; poi l'assegnamento a `c` ha a sua volta come effetto collaterale la modifica di `c`.

**Esempio.**

```
int a = 6;
int b = 12;
int c;
(c = b) = a;     // errore di compilazione
```

Il compilatore segnala un errore perché `(c = b)` è un rvalue e non un lvalue: non si può assegnargli il valore di `a`.

**Esempio** (Quanti effetti collaterali?)**.**

```
int a = 3;
int b = 4;
a = b++;
```

L'espressione `a = b++` ha due effetti collaterali: uno su `b` e uno su `a`.

## 4.2 Regola da ricordare

Un programmatore deve ricordarsi di *non modificare la stessa variabile più di una volta tra due sequence point consecutivi*.

```
i = 1;       // OK
i = i++;     // SBAGLIATO: due modifiche di i; il comportamento è indefinito
```

Poiché l'assegnamento e l'incremento nell'ultima istruzione possono avvenire in un ordine qualunque, il valore risultante di `i` è indefinito.

```
int i = i++;   // OK, qualunque cosa significhi: è la creazione di i
               // seguita dalla sua inizializzazione; il primo "=" non è
```

```
3                   // un operatore di assegnamento
```

### 4.3 Dove si trovano i sequence point

I sequence point più importanti si trovano nelle posizioni seguenti:

- alla fine di un comando (`;`);
- dopo la valutazione di tutti gli argomenti di una chiamata di funzione e prima che il controllo passi alle istruzioni della funzione;
- alla fine di un'espressione che non fa parte di un'espressione più grande: per esempio ognuna delle tre espressioni di controllo di un `for`, la condizione di un `if` o di un `while`, l'espressione di un `return` e gli inizializzatori (dopo la valutazione di 5 in `int a = 5;`);
- dopo la valutazione del primo operando di ciascuno dei seguenti operatori: `&&` (AND logico), `||` (OR logico), `?:` (condizionale) e `,` (virgola).

**Esempio.**

- L'espressione `++i < 100 ? f(i++) : (i = 0)` è ammessa, perché c'è un sequence point tra la prima modifica di `i` e quella delle altre due che viene eseguita.
- Anche `a++ && a++` è corretta (sequence point dopo il primo operando di `&&`).
- `a++ + b++` è corretta, perché ha un effetto collaterale su `a` e uno su `b`, cioè su due variabili diverse.

**Esercizio.** Che cosa stampa il programma seguente?

```c
int main()
{
    int i = 0;
    int k = i++, j = i++;
    printf("%d %d %d\n", k, j, i);
    int l = 0;
    int m = ++l, n = ++l;
    printf("%d %d %d\n", m, n, l);
    int q;
    q = ++q;
    printf("%d\n", q);
}
```

```
0 1 2
1 2 2
```

Nelle prime due dichiarazioni c'è un sequence point alla fine di ciascun inizializzatore, quindi i valori sono ben definiti. L'ultima istruzione, invece, modifica `q` due volte senza sequence point intermedio: il compilatore avverte con `warning: multiple unsequenced modifications to 'q' [-Wunsequenced]` e il comportamento è indefinito.

# 5 Riferimenti

Sul libro: sezioni 2.5, 2.6, 3.11, 3.12, 4.9, 4.10, 7.7, 10.9, appendice A, pag. 77 (espressione condizionale), pag. 87 (cast), pag. 135 (lvalue e rvalue), pag. 243 (virgola). Per effetti collaterali e sequence point:

- `https://learn.microsoft.com/it-it/cpp/c-language/side-effects?view=msvc-170`
- `https://learn.microsoft.com/it-it/cpp/c-language/c-sequence-points?view=msvc-170`